

+



Graphic Era
deemed to be **University**
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Food Surplus Management and Redistribution System

Team Name: Team #	FoodRescuers DSCPP-III-2026-T074
Team member 1 (Team Lead)	Bisht, Nidhyati – 2512510007 2512510007@geu.ac.in 
Team member 2	Gosai, Vartika – 2510370485 2510370485@geu.ac.in 

Team member 3	<i>Singh Rawat, Subhansh</i> – 2510370047 2510370047@geu.ac.in 
Team member 4	<i>Singh Mehta, Jatin –</i> 2510370090 2510370090@geu.ac.in 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Every day, vast quantities of wholesome, freshly prepared food are wasted across banquet halls, restaurants, college messes, hostel canteens, and catering functions. Meanwhile, local shelters and orphanages face persistent meal shortages every day. The issue is not that food is unavailable—it simply spoils too quickly. Hot cooked meals go bad within 2 to 4 hours, and relying on phone calls or manual coordination takes too long to arrange pickups in time. There is also no quick way to confirm whether food contains ingredients that could trigger allergic reactions or violate dietary rules. Our Food Surplus Management and Redistribution System addresses this by speeding up the matching process. Donors log their surplus items, portion sizes, expiry limits, and ingredients directly onto the platform. Rather than acting as a simple storage table, the program uses scheduling algorithms to automatically pair donations with recipient needs before items spoil.

It dynamically prioritizes incoming batches based on urgency, ensuring that meals nearing expiration are processed first. Concurrently, it cross-checks ingredient records to enforce safety gating, automatically rejecting incompatible pairings for individuals with allergies or filtering out spiced and toxic items for animal shelters. By consolidating donation registration, urgency tracking, safety verification, and recipient allocation into one streamlined process, the ultimate goal of the project is to eliminate avoidable urban food waste and channel safe nutrition to beneficiaries in need before edible food spoils.

Phase 2 Goals- The main goals of Phase 2 are listed below:-

S No.	Features	Status
1.	Integrated Web-Based Front-End Development	Completed on 05 October
2.	Relational DBMS Prototype Schema and Synthetic Data Validation	Completed on 07 October
3.	C++ Core Data Structures & OOP Class Implementation	Completed on 07 October
4.	C++ Allocation Logic, Safety Filtering and Algorithmic Unit Testing	Completed on 08 October
5.	Production Grade Relational Database (SQLite) Finalization	To be completed by 09 October
6.	End-to-End API Integration (Flask Bridge and C++ Engine Linking)	To be completed by 12 October
7.	Full System Integration and Complete Workflow Validation	To be completed by 15 October

Updated Project Approach and Architecture

The system implements a 3-tier architecture that links a responsive client frontend, a relational database management system, and an algorithmic processing core built using C++ Object-Oriented Programming and data structures.

1. Current Project Approach & System Workflow

Surplus food management is structured as a real-time, deterministic pipeline rather than static record storage:

- **Intake & Validation:** Donors register surplus meal lots with portion quantities, preparation time, remaining shelf life, and ingredient lists. Recipients register their required portions along with dietary limits.
- **Active Data Structure Scheduling:** Perishable food items are pushed into a Min-Heap priority queue ordered by remaining shelf life so batches nearest to expiry are extracted first. Recipient requests are placed into a First-In-First-Out (FIFO) queue to ensure fair order handling. In-memory active inventory is managed using a dynamic singly linked list to avoid static array limits, while a Hash Table provides fast lookups for active user profiles and ingredient tags.
- **Safety & Allergen Filtering:** Before any match is approved, the system scans the meal ingredients against the recipient's allergy tags via hash table lookups. If an allergen or dietary conflict is flagged, the match is rejected immediately.
- **Database Updates:** Once a match passes all checks, the database marks the item as allocated, subtracts the claimed quantities from active inventory, and records the transaction in the history log.

2. System Design & Object-Oriented Hierarchy

The computational core enforces core Object-Oriented Programming (OOP) paradigms:

- **Classes, Objects & Encapsulation:** Private data members represent system states, which are protected by constructors and accessed through public member functions.
- **Inheritance & Abstraction:** We built an abstract base class User to store baseline fields like names, contact numbers, and system IDs. Both Donor and NGO classes inherit from it, adding only their specific attributes.
- **Polymorphism:** Child classes override virtual methods from User. When executed, the engine calls these methods to render role-specific screens and run custom validation checks automatically.

3. Database Management System (DBMS)

We store persistent application data in SQLite tables instead of temporary variables. Structured tables track user profiles, food listings, incoming requests, and dispatch logs. We use foreign key constraints to prevent orphaned entries, while status flags track whether a batch is available or dispatched.

4. Communication Protocols, Libraries & Tools

- **Communication Protocol:** The frontend interface communicates with the backend over standard HTTP using asynchronous JSON payloads via browser fetch requests.
- **Frontend:** Developed using standard HTML, CSS, and JavaScript for user registration, surplus forms, and urgent inventory tracking etc.
- **Web Server & Database Tools:** A lightweight Python server utilizing the standard database interface library executes relational SQL queries and coordinates with the C++ execution engine.
- **C++ Standard Libraries Used:** The backend processing engine utilizes core standard headers: `<iostream>`, `<string>`, `<sstream>`, `<vector>`, `<fstream>`, `<cmath>`, `<limits>`, `<chrono>`, `<queue>` and `<algorithm>`.

Tasks Completed

Task Completed	Team Member
• Relational DBMS Initial Prototype & Schema Design: Designed an initial relational database schema using SQLite, structuring preliminary tables for users, food items, requests, and matches with primary and foreign key constraints.	Nidhyati Bisht
• Database Testing with Test Data: Wrote Python test scripts that entered test data (donors, NGOs, and food batches) into SQLite. We confirmed queries return correctly and that foreign key rules are implemented properly.	Nidhyati Bisht
• Frontend Interface Development (HTML & CSS): Built the web interface using HTML and CSS. Designed clean, responsive webpage with different sections for donor listings and NGO claims.	Jatin Singh Mehta
• Client-Side Form Validation & UI Interactivity: Added JavaScript to stop invalid form entries from reaching the backend. It checks for missing fields, positive quantities, valid expiry values, and manages UI tabs.	Jatin Singh Mehta
• C++ OOP Class Hierarchy: Structured our core C++ code around an abstract User class inherited by Donor and NGO child classes to keep logic separated.	Subhansh Singh Rawat
• Expiry-Priority Queue (Min-Heap): Implemented a Min-Heap with a custom comparator in C++. Food items sort by shelf life so that batches near to expiry always come out first.	Subhansh Singh Rawat
• FIFO Recipient Request Queue: Set up a standard First-In-First-Out queue in C++ for incoming NGO requests, ensuring claims get served by arrival order	Vartika Gosai

Challenges/Roadblocks

While implementing Phase 2, our team ran into several practical hurdles trying to connect the frontend forms, SQLite tables, and C++ engine together:

- **Choosing SQLite Over Client-Server Databases:** We initially evaluated MongoDB and MySQL alongside SQLite. We picked SQLite because our system relies on relational integrity—like tying food items to donor IDs and recipient claims through foreign keys. SQLite keeps everything in a single local file without needing to install or manage a separate background server.
- **Multi-Tier Implementation Order:** Writing the frontend, database schema, and C++ core backend logic at the same time became confusing and lead to errors in the system. We sorted this out by finishing and holding the UI development and Database development, giving us fixed data fields to work with before moving forward with the backend.
- **Validating Min-Heap Expiry Sort Order:** It was hard to verify if our heap extracted items in true order of urgency without live data. To check this, we wrote small standalone C++ test scripts with dummy timestamps, confirming that batches expiring soonest move to the front of the heap first.
- **Fair Scheduling Under High Demand:** If multiple Users ordered at the same moment, older requests could be left unattended. We implemented a strict First-In-First-Out (FIFO) queue so requests are handled strictly on first come first serve basis.
- **Dietary and Allergen Safety Filtering:** Matching donor food against NGO dietary needs without real profiles made it easy to miss false matches. We set up mock recipient records with allergen tags, then used hash tables to instantly reject any meal containing flagged ingredients(allergens).
- **Handling Unexpected Input Values:** Early program tests crashed whenever an entry had zero portions, negative shelf-life hours, or same submission times. We wrote standalone tests to catch, clean and discard such inputs before they reach the main allocation logic.

Tasks Pending

Task Pending	Team Member (to complete the task)
• Adding Production Database Rules: Moving tables into DBMS/ directory and adding CHECK constraints, foreign keys rules so invalid entries get rejected while other records stay clean. • Connecting Website to C++ via Flask: Writing backend Python routes to read incoming JSON form submissions. Building lightweight Flask routes to grab form inputs from the browser, parse the JSON, and call the compiled C++ engine with clean arguments. • End-to-End System Wiring: Linking the web frontend, Flask API layer, SQLite database, and C++ engine together so data moves across the whole system on its own. • System Exception and Error Handling: Adding safety checks across each layer so the application recovers on its own if SQLite gets locked, a process hangs, or inputs are invalid. • Full Flow User Testing: Running complete browser trials—posting a meal as a donor, letting the engine match it to an NGO, and checking that the database updates the records properly.	Nidhyati Bisht Jatin Singh Mehta Nidhyati Bisht Vartika Gosai Subhansh Singh Rawat

Project Outcome/Deliverables

The Food Surplus Management and Redistribution System delivers five interconnected components that establish an automated, reliable logistics pipeline:

- **Interactive Role based Web Interface:** A responsive frontend built with HTML, CSS, and JavaScript featuring dynamic forms, live validation, and status notifications for donors and recipient NGOs.
- **Structured Relational Database (SQLite):** A persistent storage engine enforcing table relationships, unique identifiers, and strict constraints to eliminate corrupt records and duplicate profiles.
- **C++ Computational Allocation Core:** A modular backend built on Object-Oriented principles (User base class with derived Donor and NGO structures) that couples a Min-Heap for expiry prioritization with a FIFO queue for fair chronological scheduling.
- **Instant Safety & Allergen Filter:** An automated verification module utilizing Hash Tables for rapid dietary lookups and linked lists for inventory traversal, rejecting incompatible food allocations immediately.
- **Integrated Web-to-Engine API Bridge:** A Python Flask communication layer that receives form inputs from the browser, passes data to the C++ matching engine, and updates live database records.

Together, these deliverables provide a verified platform that eliminates manual matching delays, enforces food safety, and maximizes timely surplus redistribution.

Progress Overview

The Food Surplus Management and Redistribution System is currently at approximately 75% to 80% completion for Phase 2, with core modular development tracking on schedule against initial project targets.

Development effort has focused on establishing the independent foundational layers. The role based web interface and client-side validation logic were completed early in this phase, allowing interface designs to freeze ahead of backend linking. Simultaneously, the relational SQLite prototype schema, mock data validation, and the standalone C++ algorithmic data structures (including the custom Min-Heap and FIFO request queue) have been realized on schedule.

No development tracks are currently behind baseline expectations. The remaining 20-25 % of the phase workload centers on transitioning from isolated module verification to full-pipeline execution—specifically hardening database constraints, conducting algorithmic boundary testing, constructing the Python Flask service routes, and performing complete end-to-end integration runs across all three tiers.

Codebase Information

Repository Link:- https://github.com/nidhyatibisht-2026/DSCPP-III-2026-T074_PBL_SEM3.git

Branch- main

Important Commits Information:-

1. Prototype Relational Database Schema

- Folder Name: FoodSurplusDBMS/
- Files Included: schema.sql, db_manage.py, food_surplus.db
- Description: Implemented the preliminary SQLite database prototype. Established foundational tables for accounts and food listings with primary and foreign key constraints, using synthetic data insertion scripts to verify relational table creation and baseline queries.

2. Finalized Relational Database Layer

- Folder Name: DBMS/
- Files Included: schema2.sql, db_manage2.py, food_surplus2.db
- Description: Houses the production-ready SQLite database architecture. Implements strict data integrity rules, check constraints, unique indexes, and foreign-key cascade rules, serving as the active data store for upcoming Phase 3 enhancements.

3. Client-Side User Interface & Interactivity

- Folder Name: frontend/
- Files Included: index.html, style.css, app.js
- Description: Contains our frontend webpage. We built a responsive layout with toggleable sections for donors and NGOs, using client-side JavaScript to block empty inputs and check that expiry hours and portion numbers are greater than zero.

4. Core Algorithmic Engine

- Folder Name: backend/
- Files Included: Backendforphase2.cpp
- Description: Contains our C++ allocation core. It sets up the User, Donor, and NGO classes, runs a Min-Heap to surface meals closest to expiry, and uses a FIFO queue to process incoming food requests in fair arrival order.

5. Algorithmic Testing & Verification Harness

- Folder Name: testing_debugging/
- Files Included: test_phase2.txt, Backendforphase2.cpp
- Description: Houses our test runs and bug logs. We ran tests on boundary conditions like zero shelf-life entries, duplicate timestamps, and dietary mismatches to clean up and stabilize our C++ backend logic.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Algorithmic Expiry Ordering (Min-Heap)	Pass	Verified that surplus food batches with the shortest remaining shelf-life are extracted first. Handled identical timestamps using FIFO queuing.
Dietary & Allergen Safety Filtering	Pass	Verified that dietary mismatch rules successfully block incompatible food allocations (e.g., non-veg items flagged against vegetarian NGO requirements).
Simultaneous Requests & Partial Quantities	Pass	Verified that split allocations decrement available batch stock correctly when a requested portion is smaller or larger than a single batch.
Zero / Expired Shelf-Life Handling	Pass	Fixed an initial edge case where items with 0 hours remaining entered the queue; added logic to immediately discard fully expired inventory.
Relational Database Schema & Foreign Key Constraints	Pass	Verified relational integrity in SQLite prototype; blocked orphan records and verified primary/foreign key mappings across tables.
Synthetic Data Ingestion & Query Execution	Pass	Executed test scripts inserting mock donor and NGO records to evaluate table creation, write operations, and query retrieval performance.
Client-Side Form Input Validation	Pass	Verified browser-side checks in JavaScript; successfully rejected negative quantities, missing mandatory fields, and invalid expiry dates.

Deliverables Progress

Phase 2 focused on building the core project deliverables independently: the user-facing web interface, the persistent relational database, and the C++ algorithmic matching core. The section below summarizes the current implementation status of each key deliverable as we transition into the final system integration phase.

1. Role-Based Single-Page Web Interface

- Status: Completed
- Summary: Built a single responsive webpage that switches views for donors and NGOs. It runs JavaScript checks right in the browser to catch missing fields or negative values before any form gets sent.

2. C++ Computational Allocation Core

- Status: Completed
- Summary: Implemented the matching code in C++ using object-oriented classes. It sorts food batches by remaining shelf life using a custom Min-Heap and handles NGO meal claims via an arrival-order FIFO queue.

3. Instant Safety & Allergen Filter

- Status: Completed
- Summary: Wrote hash table lookup logic that scans meal ingredients against recipient dietary restrictions, dropping unsafe pairings on the spot.

4. Structured Relational Database Layer

- Status: In Progress
- Summary: Tested the initial table design and sample data queries in SQLite; currently setting up strict check constraints, foreign keys, and indexes for the final database schema.

5. Web-to-Engine API Bridge

- Status: Pending
- Summary: Developing Python Flask route handlers to accept JSON form data from the webpage and pipe it directly into our compiled C++ program.